



Laurea Triennale in informatica-Università di Salerno
Corso di *Ingegneria del Software* - Prof. Andrea De Lucia



System Design Document CardioCare360

Riferimento	<i>2026_SDD_CardioCare360_V1.0</i>
Versione	1.0
Data	06/02/2026
Destinatario	Prof. Andrea De Lucia
Presentato da	Ciro Cannello
Approvato da	/

Indice generale

Revision History.....	3
Team Members.....	4
1. Introduzione.....	4
1.1 Scopo del sistema.....	4
1.2 Design Goals e Trade-Off.....	6
1.2.1 Design Goals.....	6
1.2.2 Design Trade-Off.....	9
1.3 Definizioni, Acronimi e Abbreviazioni.....	10
1.4 Riferimenti.....	10
1.5 Panoramica del documento.....	11
2. Architettura del Sistema Corrente.....	12
3. Architettura del Sistema Proposto (CardioCare360).....	13
3.1 Panoramica.....	13
3.2 Decomposizione in Sottosistemi.....	14
3.2.1 Diagramma dei Sottosistemi.....	17
3.2.2 Diagramma Architetture.....	19
3.3 Mapping Hardware/Software.....	20
3.3.1 Deployment Diagram.....	22
3.4 Gestione Dati Persistenti.....	23
3.4.2 Decisioni prese in fase di System Design.....	25
3.4.3 Entity Class Diagram.....	28
3.4.4 Schema Logico.....	30
3.5 Controllo Accessi e Sicurezza.....	38
3.5.1 Gestione dei ruoli.....	39
3.5.2 Matrice dei Permessi.....	39
3.5.3 Principi di sicurezza.....	40
3.6 Controllo Flusso Globale del Sistema.....	41
3.7 Boundary Conditions.....	44
3.7.1 Start up.....	44
3.7.2 Shut down.....	44
3.7.3 Failures.....	45
3.7.4 Gestione dei fallimenti.....	45
4. Servizi dei sottosistemi.....	46
4.1 Autenticazione.....	46
4.2 Gestione Appuntamenti.....	46
4.3 Gestione Disponibilità Medico.....	47
4.4 Gestione Terapie.....	47
4.5 Gestione Conversazioni.....	48
4.6 Gestione Utenti.....	48
4.7 Gestione Parametri Clinici.....	49
4.8 Gestione Appuntamento.....	49
4.9 Gestione Esami.....	50
4.10 Gestione Referti.....	50
5. Glossario.....	51

Revision History

Data	Versione	Descrizione	Autori
25/01/2026	0.1	Stesura design goals	Ciro Cannello
26/01/2026	0.2	Stesura Architettura del Sistema Corrente	Ciro Cannello
27/01/2026	0.3	Stesura dei Design Trade-Offs	Ciro Cannello
28/01/2026	0.4	Stesura della decomposizione in sottosistemi	Ciro Cannello
29/01/2026	0.5	Stesura del mapping Hardware/Software	Ciro Cannello
30/01/2026	0.6	Stesura del controllo degli accessi e sicurezza	Ciro Cannello
31/01/2026	0.7	Stesura del controllo del flusso di sistema	Ciro Cannello
01/02/2026	0.8	Stesura Boundary Conditions	Ciro Cannello
02/02/2026	0.9	Stesura della gestione Dati Persistenti	Ciro Cannello
03/02/2026	0.10	Stesura dei servizi dei Sottosistemi	Ciro Cannello
04/02/2026	0.11	Style Review	Ciro Cannello
05/02/2026	0.12	Grammar Revision	Ciro Cannello
06/02/2026	0.13	Revisione e consegna	Ciro Cannello

Team Members

Ruolo	Nome e Cognome	Acronimo	Email
PM	Ciro Cannello	CC	c.cannello@studenti.unisa.it
TM	Ciro Cannello	CC	c.cannello@studenti.unisa.it

1. Introduzione

L'obiettivo di questo documento è descrivere in modo dettagliato l'architettura software del sistema *CardioCare360*, definendo le scelte progettuali, i componenti principali, le interazioni tra i moduli e le linee guida che orientano l'implementazione. Lo SDD fornisce una visione tecnica completa del sistema, traducendo i requisiti funzionali e non funzionali — già definiti nel RAD — in un'architettura concreta, modulare e scalabile.

Il documento è rivolto a sviluppatori, architetti software, membri del team tecnico e a tutte le figure coinvolte nella progettazione e realizzazione del sistema. La sua funzione è garantire coerenza, tracciabilità e chiarezza durante l'intero ciclo di sviluppo, fornendo una base solida per l'implementazione, i test e la futura manutenzione.

1.1 Scopo del sistema

CardioCare360 è una piattaforma digitale progettata per supportare la gestione completa del percorso cardiologico, facilitando l'interazione tra pazienti, medici e amministratori. Il sistema consente:

- la gestione degli appuntamenti cardiologici
- la consultazione e l'aggiornamento dei referti
- la comunicazione strutturata tra paziente e medico
- la gestione delle disponibilità dei medici
- il monitoraggio delle attività amministrative
- la conservazione sicura dei dati clinici

Lo scopo principale del sistema è **ottimizzare il flusso informativo**, ridurre gli errori operativi e migliorare l'efficienza del processo clinico, garantendo al tempo stesso sicurezza, affidabilità e conformità alle normative vigenti.

Il sistema è progettato per essere:

- **scalabile**, così da supportare un numero crescente di utenti
- **modulare**, per facilitare l'estensione futura
- **sicuro**, con particolare attenzione alla protezione dei dati sanitari
- **usabile**, grazie a un'interfaccia chiara e coerente con il dominio medico

Questa sezione definisce quindi il perimetro funzionale del sistema e stabilisce le basi concettuali per le scelte architetture descritte nei capitoli successivi.

1.2 Design Goals e Trade-Off

1.2.1 Design Goals

Rank	ID	Design Goal	Descrizione	Categoria	RNF
1	DG_1	Funzionalità	Il sistema deve garantire che tutte le funzionalità con priorità “Alta” siano presenti e operative nel primo rilascio.	Functionality	RNF_F1
2	DG_2	Usabilità	Il 98% degli utenti deve poter utilizzare la piattaforma in modo efficace senza necessità di assistenza o documentazione.	Usabilità	RNF_U1
3	DG_3	Usabilità	L’interfaccia deve essere semplice, intuitiva e coerente, garantendo un’interazione rapida e priva di ambiguità.	Usabilità	RNF_U2
4	DG_4	Usabilità	Le interfacce devono essere correttamente visualizzabili sul 90% dei dispositivi mobile più diffusi.	Usabilità	RNF_U3
5	DG_5	Affidabilità	Il sistema deve garantire una disponibilità del servizio pari ad almeno il 99%, prevedendo eventuali interventi di manutenzione programmata .	Affidabilità	RNF_A1
6	DG_6	Affidabilità	Il sistema deve gestire correttamente almeno il 90% delle operazioni errate effettuate dall’utente.	Affidabilità	RNF_A2
7	DG_7	Sicurezza	L’applicazione deve garantire autenticazione sicura su connessione cifrata, proteggendo i dati sensibili da accessi non autorizzati.	Sicurezza	RNF_A3

Rank	ID	Design Goal	Descrizione	Categoria	RNF
8	DG_8	Prestazioni	Il sistema deve supportare la gestione di almeno 50 utenti connessi contemporaneamente, mantenendo il corretto funzionamento delle principali funzionalità della piattaforma .	Prestazioni	RNF_P1
9	DG_9	Prestazioni	Il sistema deve garantire tempi di risposta rapidi e un'elevata capacità di memorizzazione dei dati.	Prestazioni	RNF_P2
10	DG_10	Sostenibilità	Il sistema deve essere progettato secondo una struttura modulare che favorisca il riutilizzo dei componenti e faciliti l'estensione futura delle funzionalità.	Sostenibilità	RNF_S1
11	DG_11	Scalabilità	L'architettura hardware deve essere scalabile per supportare un aumento del carico e degli utenti.	Sostenibilità	RNF_S2
12	DG_12	Prestazioni	Il sistema deve fornire una risposta entro 1 secondo nel 95% delle richieste.	Implementazione	RNF_IM1
13	DG_13	Implementazione	L'interfaccia utente deve utilizzare tecnologie web standard (HTML, CSS, JavaScript).	Implementazione	RNF_IM2
14	DG_14	Compatibilità	Il sistema deve essere accessibile da dispositivi iOS e Android nelle versioni più aggiornate.	Implementazione	RNF_IM3
15	DG_15	Interfaccia	Il sistema deve essere in grado di generare documenti in formato PDF o equivalenti.	Interfaccia	RNF_IN1

Rank	ID	Design Goal	Descrizione	Categoria	RNF
16	DG_16	Packaging	Il sistema deve essere distribuito come applicazione web, accessibile tramite browser senza installazione locale.	Packaging	RNF_PA1
17	DG_17	Packaging	La piattaforma deve essere fruibile da dispositivi iOS e Android tramite browser compatibile.	Packaging	RNF_PA2
18	DG_18	Deployment	Il sistema deve essere rilasciato in un ambiente server configurato con tutte le dipendenze necessarie (Java, React build, DB).	Packaging	RNF_PA3
19	DG_19	Architettura	Il pacchetto software deve essere organizzato in moduli separati (frontend, backend, database).	Packaging	RNF_PA4
20	DG_20	Legali	Il sistema deve essere distribuito con licenza proprietaria che tuteli il codice sorgente.	Legali	RNF_LE1
21	DG_21	Privacy	Il sistema deve garantire conformità al GDPR (Regolamento UE 2016/679).	Legali	RNF_LE2
22	DG_22	Privacy	I dati sanitari devono essere conservati e trattati nel rispetto delle normative, accessibili solo al personale autorizzato.	Legali	RNF_LE3

1.2.2 Design Trade-Off

Trade-Off	Descrizione
Sicurezza vs Tempi di risposta	L'adozione di meccanismi di autenticazione e protezione dei dati (DG_7) può introdurre un overhead nell'elaborazione delle richieste. Tale scelta è necessaria per garantire la protezione dei dati sensibili, mantenendo comunque l'obiettivo di fornire una risposta entro 1 secondo nel 95% delle richieste (DG_12).
Robustezza vs Tempi di risposta	La validazione degli input e la gestione degli errori (DG_6) richiedono controlli aggiuntivi sulle richieste degli utenti. Tali controlli possono aumentare leggermente il tempo di elaborazione, ma sono necessari per garantire l'affidabilità del sistema.
Riutilizzabilità vs Specializzazione	L'organizzazione del software in moduli separati e il riutilizzo dei componenti (DG_10 e DG_19) favoriscono la manutenibilità e le future estensioni del sistema. Questa scelta può richiedere una maggiore generalizzazione dei componenti rispetto alla realizzazione di soluzioni specifiche per singole funzionalità.
Portabilità vs Complessità dell'interfaccia	Il supporto a dispositivi desktop e mobile (DG_4, DG_14 e DG_17) richiede un'interfaccia responsive e compatibile con differenti dimensioni dello schermo e browser. Questo può aumentare la complessità della progettazione dell'interfaccia, ma permette di rendere il sistema accessibile da un numero maggiore di dispositivi.

1.3 Definizioni, Acronimi e Abbreviazioni

Acronimi:

- **RAD** – Requirement Analysis Document
- **DG** – Design Goal
- **RNF** – Requisito Non Funzionale
- **UC** – Use Case
- **PAZ** – Paziente
- **MED** – Medico
- **ADS** – Amministratore di Sistema
- **RDBMS** – Relational Database Management System

1.4 Riferimenti

Libri

- *Object-Oriented Software Engineering: Using UML, Patterns, and Java*, B. Bruegge & A.H. Dutoit, Prentice Hall.

Documentazione Tecnica Ufficiale

- **Java (Oracle)** – Documentazione ufficiale del linguaggio e delle API.
- **Spring Framework** – Documentazione ufficiale per Spring Boot, Spring MVC e Spring Security.
- **React (Meta)** – Documentazione ufficiale per lo sviluppo frontend.
- **Bootstrap** – Documentazione ufficiale del framework CSS per layout responsive.
- **HTML / CSS / JavaScript (MDN Web Docs)** – Documentazione ufficiale per le tecnologie web.
- **RDBMS utilizzato MySQL** – Documentazione ufficiale del database scelto.

Standard e Linee Guida

- **UML 2.x Specification** – Object Management Group (OMG).
- **GDPR – Regolamento UE 2016/679** – Normativa sulla protezione dei dati personali.

1.5 Panoramica del documento

Il presente documento di *System Design* è articolato in cinque sezioni principali:

- **Introduzione** Descrive lo scopo del sistema, il contesto applicativo, gli attori coinvolti e gli obiettivi di design (Design Goals) che il sistema intende soddisfare.
- **Architettura software corrente** Presenta lo stato attuale dell'architettura del sistema, evidenziando le componenti già esistenti, le loro responsabilità e le eventuali limitazioni che motivano la nuova progettazione.
- **Architettura software proposta** Definisce la nuova architettura del sistema, descrivendo la suddivisione in sottosistemi, il mapping Hardware/Software, la gestione dei dati persistenti, le interfacce tra i componenti e le *boundary conditions* che regolano il comportamento globale del sistema.
- **Servizi dei sottosistemi** Descrive i servizi forniti dai principali sottosistemi del sistema CardioCare360 e le relative responsabilità
- **Glossario** Contiene la lista dei termini tecnici, acronimi e abbreviazioni utilizzati nel documento, con relativa spiegazione.

2. Architettura del Sistema Corrente

Nel contesto considerato, la gestione delle informazioni e delle attività cardiologiche può avvalersi di diversi sistemi digitali, tra cui il Fascicolo Sanitario Elettronico (FSE), i portali regionali di prenotazione (CUP) e le piattaforme ospedaliere per la consultazione dei referti. Tali sistemi forniscono funzionalità specifiche, ma non costituiscono necessariamente un'unica piattaforma integrata per la gestione dell'intero percorso cardiologico.

In particolare, le funzionalità relative a:

- gestione delle disponibilità dei medici cardiologi;
- prenotazione delle visite;
- gestione e consultazione dei referti;
- comunicazione tra paziente e medico;
- monitoraggio dei parametri clinici;
- gestione del percorso clinico del paziente;

possono risultare distribuite tra strumenti e sistemi differenti.

L'architettura interna dei sistemi sanitari esistenti non è analizzabile direttamente nell'ambito del presente progetto, in quanto non sono disponibili il relativo codice sorgente né la documentazione tecnica interna. Pertanto, la descrizione del sistema corrente si limita alle funzionalità osservabili e al contesto operativo di riferimento.

Questa situazione evidenzia la necessità di una soluzione integrata che riunisca in un unico sistema le principali funzionalità necessarie alla gestione del percorso cardiologico.

CardioCare360 viene pertanto progettato ex-novo, senza una migrazione diretta da un'architettura software preesistente. L'architettura proposta, descritta nel capitolo successivo, è stata definita sulla base dei requisiti funzionali e non funzionali individuati nel RAD.

3. Architettura del Sistema Proposto (CardioCare360)

3.1 Panoramica

L'architettura proposta per CardioCare360 adotta un modello Three-Tier, articolato nei seguenti livelli:

- Presentation Layer → React + Bootstrap
- Application Layer → Spring Boot (Java)
- Data Layer → MySQL

La separazione in tre livelli consente di distribuire le responsabilità del sistema tra componenti distinti e favorisce:

- separazione delle responsabilità;
- manutenibilità;
- riuso dei componenti;
- scalabilità;
- sicurezza tramite Spring Security;
- disaccoppiamento tra frontend e backend tramite API REST.

Il frontend comunica con il backend mediante API REST. Il backend implementa la logica applicativa e accede ai dati persistenti tramite Spring Data JPA, che gestisce il mapping tra le entità Java e il database relazionale MySQL.

L'architettura consente pertanto di mantenere separata la presentazione dalla logica applicativa e dalla gestione della persistenza, facilitando l'evoluzione e la manutenzione del sistema.

3.2 Decomposizione in Sottosistemi

La decomposizione deriva direttamente dagli Activity Diagram di Paziente, Medico e Amministratore (già completati).

1. Sottosistema Autenticazione e Sicurezza

Responsabile di:

- registrazione del Paziente;
- autenticazione degli utenti;
- login e logout;
- gestione dell'autorizzazione in base al ruolo;
- generazione e verifica dei token JWT;
- protezione delle risorse tramite Spring Security;
- gestione delle informazioni dell'utente autenticato.

Tecnologie: Spring Boot, Spring Security, JWT.

2. Sottosistema Gestione Utenti

Responsabile di:

- gestione dei Pazienti da parte dell'Amministratore;
- gestione dei Medici da parte dell'Amministratore;
- creazione di nuovi utenti;
- visualizzazione dei dati degli utenti;
- modifica dei dati dei Pazienti e dei Medici;
- eliminazione degli utenti secondo i permessi previsti;
- gestione dei profili personali.

Tecnologie: Spring Boot, Spring Data JPA, React.

3. Sottosistema Gestione Appuntamenti

Responsabile di:

- prenotazione delle visite;
- visualizzazione degli appuntamenti;
- modifica degli appuntamenti;
- annullamento degli appuntamenti;
- conferma e gestione degli appuntamenti da parte del Medico;

- visualizzazione del dettaglio dell'appuntamento.

Tecnologie: Spring Boot REST, React, MySQL.

4. Sottosistema Gestione Disponibilità

Responsabile di:

- inserimento delle disponibilità del Medico;
- modifica delle disponibilità;
- eliminazione delle disponibilità;
- gestione degli slot orari;
- visualizzazione delle disponibilità utilizzabili per le prenotazioni.

Tecnologie: Spring Boot, Spring Data JPA, React.

5. Sottosistema Gestione Esami

Responsabile di:

- prenotazione degli esami diagnostici;
- visualizzazione degli esami da parte del Paziente;
- visualizzazione degli esami da refertare da parte del Medico;
- gestione delle informazioni relative all'esame;
- refertazione dell'esame da parte del Medico.

Tecnologie: Spring Boot REST, Spring Data JPA, React.

6. Sottosistema Gestione Referti e Documenti

Responsabile di:

- caricamento dei referti;
- associazione del referto all'esame/visita;
- visualizzazione dei referti;
- gestione dei documenti PDF;
- download dei documenti;
- memorizzazione e recupero dei file associati ai referti.

Tecnologie: Spring Boot, Spring Data JPA, gestione dei file PDF.

Qui ho volutamente mantenuto **Referto e DocumentoPDF** nello stesso sottosistema, perché nel codice sono presenti entrambi e hanno responsabilità strettamente collegate.

7. Sottosistema Gestione Parametri Clinici

Responsabile di:

- inserimento dei parametri clinici da parte del Paziente;
- visualizzazione dei parametri;
- gestione dello storico dei parametri;
- visualizzazione dei parametri da parte del Medico;
- gestione delle soglie dei parametri;
- individuazione di eventuali valori fuori soglia.

Tecnologie: Spring Boot, Spring Data JPA, React.

8. Sottosistema Terapie

Responsabile di:

- creazione delle terapie da parte del Medico;
- gestione delle terapie;
- visualizzazione delle terapie da parte del Paziente;
- visualizzazione delle terapie da parte del Medico;
- gestione delle informazioni relative ai farmaci e alla terapia.

Tecnologie: Spring Boot, Spring Data JPA, React.

9. Sottosistema Comunicazione e Notifiche

Responsabile di:

- gestione delle conversazioni tra Paziente e Medico;
- invio e ricezione dei messaggi;
- visualizzazione delle conversazioni;
- gestione delle notifiche;
- visualizzazione degli avvisi relativi alle attività del sistema.

Tecnologie: Spring Boot REST, Spring Data JPA, React.

10. Sottosistema Persistenza

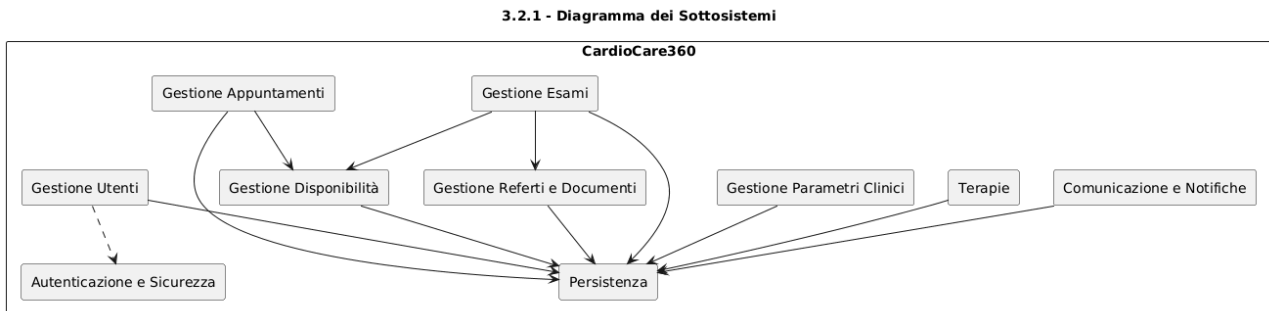
Responsabile di:

- gestione delle entità persistenti;
- accesso ai dati tramite repository;
- mapping tra entità Java e database relazionale;

- esecuzione delle operazioni CRUD;
- gestione delle transazioni;
- persistenza dei dati applicativi.

Tecnologie: Spring Data JPA, Hibernate, MySQL.

3.2.1 Diagramma dei Sottosistemi



Il sistema CardioCare360 è organizzato in una serie di sottosistemi funzionali, ciascuno responsabile di un insieme coerente di operazioni legate al dominio cardiologico.

I sottosistemi applicativi utilizzano il livello di persistenza per la memorizzazione e il recupero dei dati, attraverso i meccanismi di accesso ai dati forniti da Spring Data JPA. Questa organizzazione consente di mantenere separata la logica applicativa dalla gestione del database.

Il sottosistema Persistenza è implementato tramite una componente COTS (Commercial Off-The-Shelf),

costituita dal sistema di gestione di basi di dati relazionali MySQL, utilizzato per la memorizzazione delle informazioni relative a utenti, appuntamenti, disponibilità, esami, referti, parametri clinici, terapie e comunicazioni interne al sistema.

I sottosistemi applicativi sono realizzati secondo una struttura architetturale coerente con il modello adottato, basata sui seguenti elementi principali:

- GUI:

contiene le view e i componenti React che costituiscono l'interfaccia

utente, incluse le dashboard di Paziente, Medico e Amministratore.

- Controller:

espone le API REST e gestisce le richieste provenienti dal frontend, delegando l'elaborazione ai servizi applicativi.

- Service:

implementa la logica di business associata alle funzionalità dei diversi sottosistemi, quali prenotazioni, disponibilità, esami, referti, parametri clinici, terapie e gestione degli utenti.

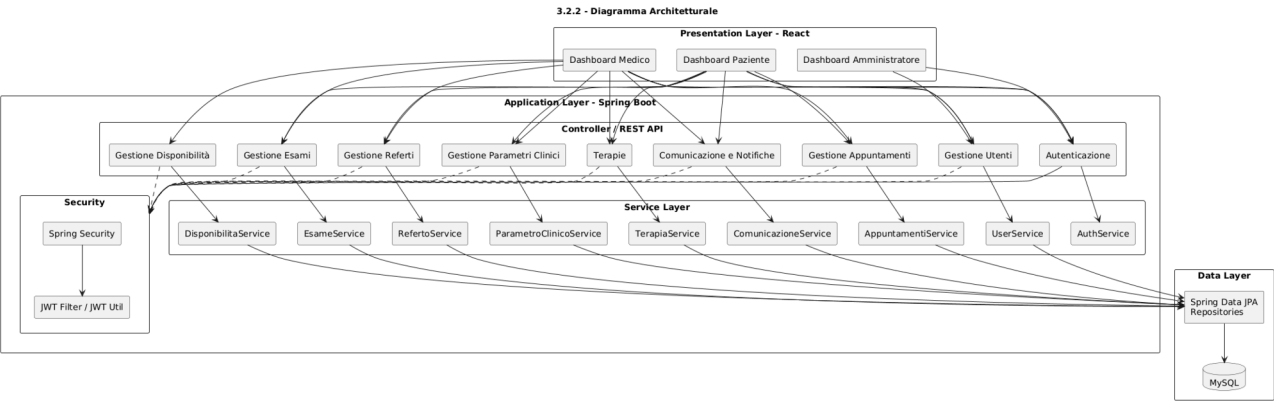
- Entity:

rappresenta i principali oggetti del dominio persistiti nel database e mappati tramite JPA.

A questi componenti si aggiungono i meccanismi specifici del sottosistema di Autenticazione e Sicurezza, che gestisce l'autenticazione e l'autorizzazione degli utenti mediante Spring Security e token JWT.

Questa organizzazione consente una chiara separazione delle responsabilità, facilita la manutenibilità del sistema e favorisce l'evoluzione indipendente delle diverse funzionalità.

3.2.2 Diagramma Architeturale



3.3 Mapping Hardware/Software

Il sistema CardioCare360 adotta un'architettura client-server basata su tre livelli principali: Presentation Layer, Application Layer e Data Layer.

Il mapping tra componenti software e infrastruttura hardware è il seguente.

Client – Presentation Layer

Il client è costituito da un dispositivo dotato di un browser Web moderno e di una connessione alla rete. Il sistema può essere utilizzato da PC desktop, laptop, tablet e smartphone.

Sul client viene eseguita l'applicazione frontend sviluppata in **React**, responsabile della gestione dell'interfaccia grafica e dell'interazione con il backend attraverso richieste HTTP/HTTPS.

Le interfacce sono differenziate in base al ruolo dell'utente e comprendono le funzionalità disponibili per **Paziente, Medico e Amministratore**.

Server Backend – Application Layer

Il backend del sistema è eseguito sul componente hardware denominato **ServerBackend**. Esso può essere realizzato mediante un server fisico, una macchina virtuale oppure un ambiente di hosting.

Sul ServerBackend è eseguita l'applicazione sviluppata in **Java con Spring Boot**, che espone le funzionalità del sistema attraverso API REST.

Il backend comprende:

- Controller REST per la gestione delle richieste provenienti dal frontend;
- Service per l'implementazione della logica applicativa;
- meccanismi di autenticazione e autorizzazione;
- **Spring Security e JWT** per la gestione della sicurezza;
- componenti Repository basati su Spring Data JPA per l'accesso ai dati.

Database Server – Data Layer

Il livello dati è costituito dal DBMS relazionale **MySQL**, responsabile della persistenza delle informazioni del sistema.

Il database contiene i dati relativi, tra gli altri, a:

- utenti e ruoli;
- appuntamenti;
- disponibilità dei medici;
- esami;
- referti e documenti associati;
- parametri clinici;
- terapie;
- notifiche e comunicazioni.

L'accesso al database viene effettuato dal backend tramite **Spring Data JPA**, che permette di gestire la persistenza delle entità applicative.

Comunicazione Client-Server

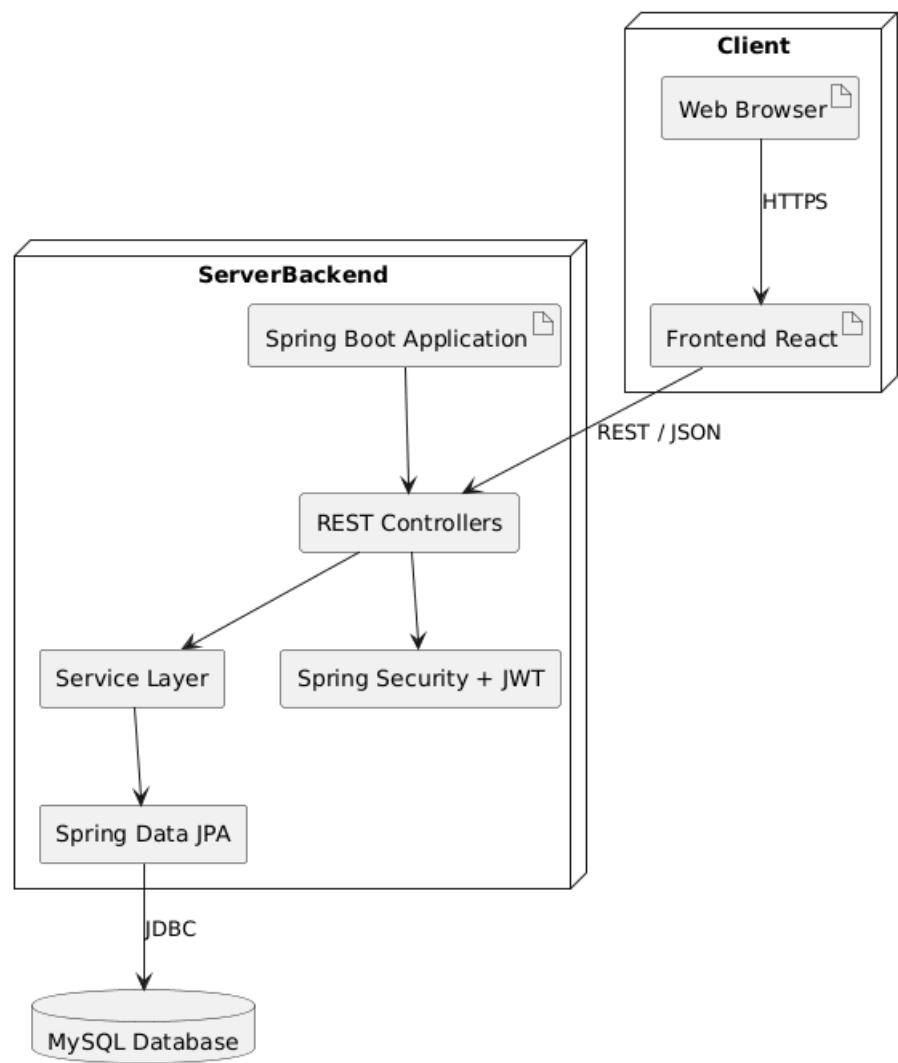
La comunicazione tra il frontend React e il backend Spring Boot avviene tramite **HTTP/HTTPS**, utilizzando API REST e messaggi in formato **JSON**.

Il client invia una richiesta al ServerBackend; il backend autentica e autorizza la richiesta, esegue la relativa logica applicativa, accede eventualmente ai dati persistenti tramite Spring Data JPA e restituisce al client una risposta JSON.

L'utilizzo di HTTPS contribuisce a garantire la confidenzialità e l'integrità delle comunicazioni tra client e server.

3.3.1 Deployment Diagram

3.3.1 - Deployment Diagram



3.4 Gestione Dati Persistenti

3.4.1 Introduzione

Il sistema CardioCare360 utilizza un database relazionale MySQL, scelto per la sua affidabilità, la compatibilità con Spring Data JPA e l'integrazione con l'architettura Three-Tier del sistema.

L'adozione di un RDBMS garantisce:

- flessibilità nella modellazione dei dati;
- utilizzo di query SQL standardizzate;
- gestione dell'accesso concorrente ai dati;
- supporto alle proprietà ACID;
- affidabilità e consistenza dei dati;
- integrazione con il livello applicativo attraverso Spring Data JPA.

Il modello dati è stato progettato seguendo criteri di normalizzazione, riduzione della ridondanza e coerenza con il dominio applicativo cardiologico.

L'accesso ai dati persistenti è gestito dal backend Spring Boot tramite Spring Data JPA, che permette di astrarre le operazioni di accesso al database attraverso repository e di effettuare il mapping tra le entità applicative e le corrispondenti strutture del database.

3.4.2 Decisioni prese in fase di System Design

Durante la fase di System Design è stata effettuata un'analisi del dominio cardiologico e dei principali flussi operativi del sistema.

Da questa analisi sono emerse le principali scelte progettuali relative al modello dei dati, basate sulle entità previste dallo schema SQL del sistema.

Entità Anagrafiche

utente

Rappresenta l'identità di accesso al sistema. Gestisce credenziali, ruoli e informazioni di base.

paziente

Contiene i dati clinici e anagrafici del paziente. È collegato ad appuntamenti, visite, referti, parametri clinici e terapie.

medico

Rappresenta il professionista sanitario. È collegato a disponibilità, visite, referti, appuntamenti e conversazioni.

amministratore

Rappresenta l'utente responsabile della supervisione del sistema e delle operazioni amministrative.

Entità Appuntamenti e Visite

appuntamento

appuntamento — Modella il ciclo di vita della prenotazione tra paziente e medico, incluso il tipo di visita richiesta (es. visita cardiologica, controllo), la data, l'ora e lo stato dell'appuntamento.

disponibilita_medico

Permette ai medici di definire le fasce orarie disponibili per le prenotazioni.

slot_orario

Rappresenta gli slot temporali prenotabili associati alle disponibilità del medico.

Entità Esami e Referti

esame

Contiene le informazioni relative agli esami diagnostici, tra cui tipologia, data, medico e paziente.

referto

Rappresenta il referto clinico associato a un esame o a una visita. Contiene i relativi metadati e i collegamenti con medico e paziente.

documento_pdf

Gestisce il documento PDF associato al referto, mantenendo separati i dati del referto dai dati relativi al documento.

Entità Terapie

terapia

Contiene le informazioni relative alle terapie assegnate al paziente. Collega medico, paziente, durata, note e farmaci associati.

farmaco

Rappresenta i farmaci associati alle terapie, permettendone il riutilizzo nelle diverse prescrizioni.

Entità Parametri Clinici

parametro

Definisce la tipologia di parametro monitorato dal sistema, ad esempio pressione, glicemia o saturazione. Contiene informazioni quali nome, unità di misura e descrizione ed è utilizzato come riferimento per i parametri clinici monitorati.

parametro_clinico

Contiene i valori clinici rilevati per il paziente e li collega alla tipologia di parametro corrispondente, consentendone il monitoraggio nel tempo.

storico_parametri

Traccia l'evoluzione dei valori dei parametri clinici nel tempo, consentendo di analizzarne l'andamento.

soglia_parametro

Definisce le soglie associate ai parametri clinici e supporta l'individuazione di condizioni che richiedono attenzione.

Entità Comunicazione

messaggio

Gestisce la comunicazione tra paziente e medico. Contiene il testo del messaggio, il timestamp, il mittente e il destinatario.

conversazione

Raggruppa i messaggi relativi a una conversazione tra gli utenti, permettendo di mantenere l'ordine e la tracciabilità delle comunicazioni.

Conclusione

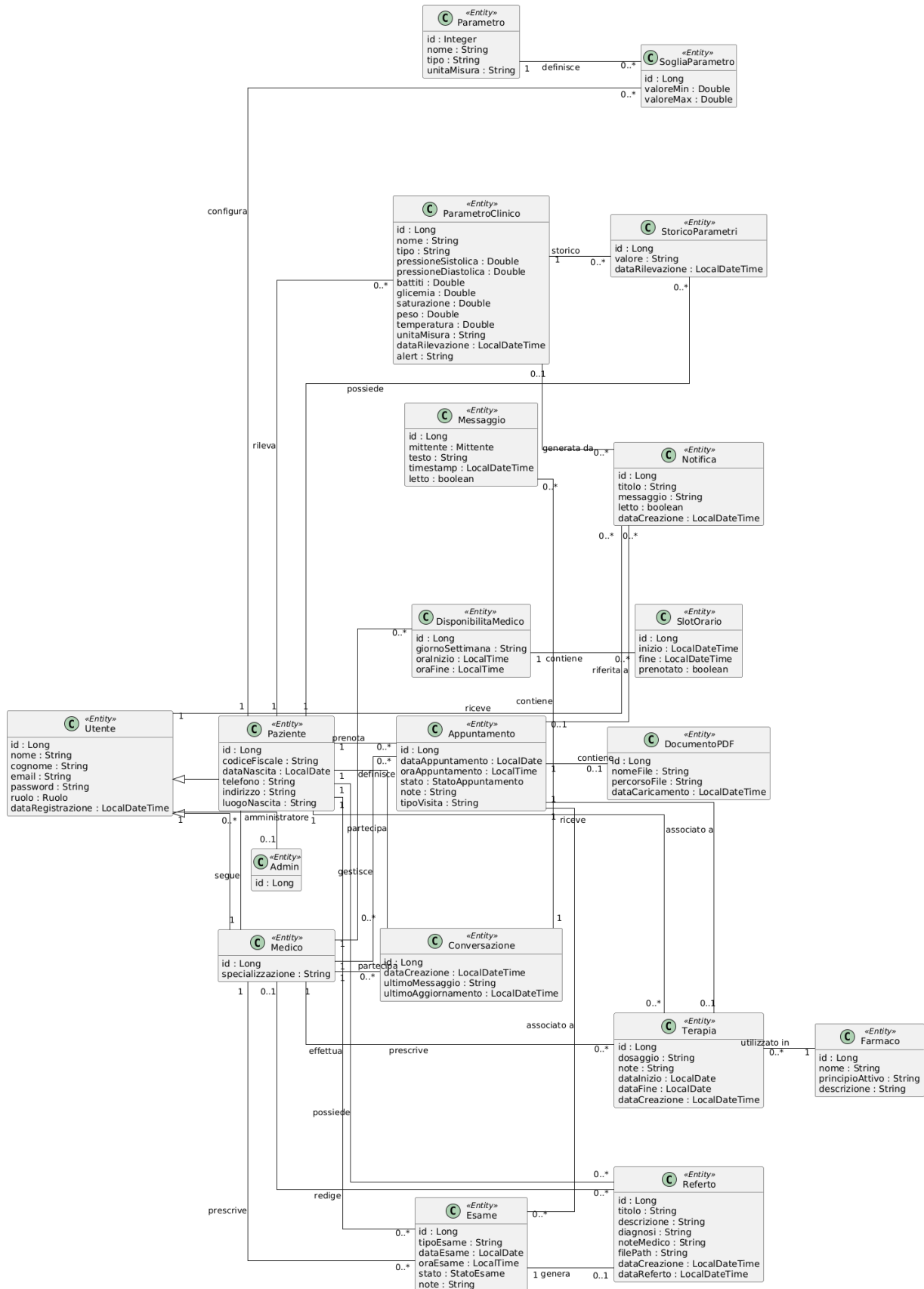
Il modello dati di CardioCare360 è:

- coerente con lo schema SQL del sistema;**
- coerente con il dominio cardiologico;**
- progettato per ridurre le ridondanze;**
- strutturato secondo criteri di normalizzazione;**
- estensibile;**
- integrato con l'architettura Three-Tier;**
- progettato per supportare i principali flussi clinici e amministrativi del sistema.**

La struttura adottata favorisce la consistenza dei dati, la manutenibilità del sistema e l'estensione futura delle funzionalità.

3.4.3 Entity Class Diagram

CardioCare360 - Entity Class Diagram (DB Real Alignment)



3.4.4 Schema Logico

Lo schema logico del database rappresenta la traduzione del modello concettuale in un modello relazionale implementabile in MySQL. In questa sezione vengono descritte le principali tabelle del sistema, i relativi attributi, le chiavi primarie, le chiavi esterne e le relazioni tra le entità.

Utente

Contiene le informazioni di base relative agli utenti del sistema e ne identifica il ruolo applicativo.

- Chiave primaria: `id`
- Attributi:
 - `nome (varchar(50))`
 - `cognome (varchar(50))`
 - `email (varchar(100), unico)`
 - `password (varchar(255))`
 - `ruolo (enum: PAZIENTE, MEDICO, ADMIN)`
 - `data_registrazione (timestamp)`
- Relazioni:
 - 1–1 con `Paziente`
 - 1–1 con `Medico`
 - 1–1 con `Admin`
 - 1–N con `Notifica`

La struttura originale dello SDD definisce correttamente `Utente` come entità comune e indica le relazioni con i tre ruoli e con le notifiche.

Paziente

Rappresenta un utente con ruolo di paziente e contiene le relative informazioni anagrafiche.

- Chiave primaria: `id`
- Chiave esterna: `id → Utente(id)`
- Attributi:
 - `codice_fiscale (varchar(16), unico)`
 - `data_nascita (date)`
 - `telefono (varchar(20))`
 - `indirizzo (varchar(255))`

- `luogo_nascita (varchar(100))`
- `medico_id → Medico(id)`
- Relazioni:
 - N-1 con `Medico`
 - 1-N con `Appuntamento`
 - 1-N con `Esame`
 - 1-N con `Terapia`
 - 1-N con `ParametroClinico`
 - 1-N con `Referto`
 - 1-N con `Conversazione`
 - 1-N con `StoricoParametri`

Questi attributi e collegamenti sono quelli riportati nello schema logico dello SDD.

Medico

Rappresenta un utente con ruolo di medico.

- Chiave primaria: `id`
- Chiave esterna: `id → Utente(id)`
- Attributi:
 - `specializzazione (varchar(100))`
- Relazioni:
 - 1-N con `Appuntamento`
 - 1-N con `Esame`
 - 1-N con `Terapia`
 - 1-N con `DisponibilitaMedico`
 - 1-N con `Conversazione`
 - 1-N con `Referto`
 - 1-N con `SogliaParametro`

Questa struttura è coerente con la descrizione già presente nello SDD.

Admin

Rappresenta un utente con ruolo amministrativo e permette di distinguere gli account amministrativi dagli altri utenti.

- Chiave primaria: `id`
- Chiave esterna: `id` → `Utente(id)`
- Attributi specifici: nessuno

Nota: la tabella `amministratore` include nello schema fisico del database anche il campo `livello`, non riportato in questo modello logico/Entity Class in quanto non utilizzato dalla logica applicativa del sistema attuale. Il campo è mantenuto nello schema fisico come possibile estensione futura (es. livelli di privilegio differenziati per gli amministratori)

Appuntamento

Rappresenta una prenotazione tra paziente e medico.

- Chiave primaria: `id`
- Chiavi esterne:
 - `paziente_id` → `Paziente(id)`
 - `medico_id` → `Medico(id)`
- Attributi:
 - `data_appuntamento` (date)
 - `ora_appuntamento` (time)
 - `stato` (varchar(20))
 - `note` (varchar(255))
 - `tipo_visita` (varchar(255))
- Relazioni:
 - 1–1 con `DocumentoPDF`
 - 1–N con `Esame`
 - 1–N con `Notifica`
 - 1–0..1 con `Terapia`

La precedente versione riportava erroneamente la relazione con `Terapia` come 1–1; nel diagramma Entity Class è invece modellata come 1–0..1.

DocumentoPDF

Rappresenta un documento PDF associato a un appuntamento.

- Chiave primaria: `id`
- Chiave esterna:
 - `appuntamento_id` → `Appuntamento(id)`
- Attributi:
 - `nome_file` (`varchar(255)`)
 - `percorso` (`varchar(255)`)
 - `data_caricamento` (`timestamp`)

Esame

Rappresenta un esame clinico prenotato o effettuato.

- Chiave primaria: `id`
- Chiavi esterne:
 - `paziente_id` → `Paziente(id)`
 - `medico_id` → `Medico(id)`
 - `appuntamento_id` → `Appuntamento(id)`
- Attributi:
 - `tipo_esame` (`varchar(255)`)
 - `data_esame` (`date`)
 - `ora_esame` (`time`)
 - `stato` (`varchar(50)`)
 - `note_paziente` (`text`)
 - `note_medico` (`text`)
 - `note` (`text`)
- Relazioni:
 - 1-1 con `Referto`

Referto

Rappresenta il referto medico associato a un esame.

- Chiave primaria: `id`
- Chiavi esterne:
 - `esame_id` → `Esame(id)`
 - `paziente_id` → `Paziente(id)`
 - `medico_id` → `Medico(id)`
- Attributi:
 - `data_referto` (datetime)
 - `descrizione` (text)
 - `diagnosi` (varchar(500))
 - `file_path` (varchar(255))
 - `titolo` (varchar(255))
 - `note_medico` (text)

Terapia

Rappresenta una terapia prescritta dal medico al paziente.

- Chiave primaria: `id`
- Chiavi esterne:
 - `paziente_id` → `Paziente(id)`
 - `medico_id` → `Medico(id)`
 - `farmaco_id` → `Farmaco(id)`
 - `appuntamento_id` → `Appuntamento(id)`
- Attributi:
 - `data_inizio` (date)
 - `data_fine` (date)
 - `dosaggio` (varchar(255))
 - `note` (varchar(500))
 - `data_creazione` (datetime)

Farmaco

Rappresenta un farmaco utilizzato nelle terapie.

- Chiave primaria: `id`

- **Attributi:**
 - nome (varchar(100))
 - descrizione (varchar(255))
 - principio_attivo (varchar(100))
- **Relazioni:**
 - 1–N con Terapia

Parametro

Rappresenta la tipologia di parametro clinico monitorato dal sistema.

- Chiave primaria: id
- **Attributi:**
 - nome (varchar(100))
 - unita_misura (varchar(50))
 - descrizione (varchar(255))
- **Relazioni:**
 - 1–N con ParametroClinico
 - 1–N con StoricoParametri
 - 1–N con SogliaParametro
 - 1–N con Notifica

Questa è la struttura testuale attualmente presente nel documento dello SDD.

ParametroClinico

Rappresenta un valore clinico rilevato per un paziente in una specifica sessione di misurazione.

- Chiave primaria: id
- Chiave esterna: paziente_id → Paziente(id)
- Attributi: tipo (varchar), battiti (double), glicemia (double), pressione_sistolica (double), pressione_diastolica (double), saturazione (double), peso (double), temperatura (double), data_rilevazione (datetime), alert (varchar)
- Relazioni: 1–N con StoricoParametri

StoricoParametri

Traccia la cronologia dei valori clinici registrati.

- Chiave primaria: `id`
- Chiavi esterne:
 - `paziente_id` → `Paziente(id)`
 - `parametro_id` → `Parametro(id)`
- Attributi:
 - `valore` (`varchar(50)`)
 - `data_rilevazione` (`timestamp`)

SogliaParametro

Rappresenta le soglie di allerta definite per un determinato parametro clinico.

- Chiave primaria: `id`
- Chiavi esterne:
 - `parametro_id` → `Parametro(id)`
 - `paziente_id` → `Paziente(id)`
- Attributi:
 - `valore_min` (`varchar(50)`)
 - `valore_max` (`varchar(50)`)

Notifica

Rappresenta una notifica associata a un utente e, quando previsto, a un appuntamento o a un parametro.

- Chiave primaria: `id`
- Chiavi esterne:
 - `utente_id` → `Utente(id)`
 - `appuntamento_id` → `Appuntamento(id)`
 - `parametro_id` → `Parametro(id)`
- Attributi:
 - `messaggio` (`varchar(1000)`)
 - `tipo` (`enum: INFO, AVVISO, URGENTE`)
 - `data_invio` (`timestamp`)
 - `letto` (`tinyint`)

- `data_creazione (datetime)`
- `titolo (varchar(255))`

Conversazione

Rappresenta una conversazione tra paziente e medico.

- Chiave primaria: `id`
- Chiavi esterne:
 - `paziente_id` → `Paziente(id)`
 - `medico_id` → `Medico(id)`
- Attributi:
 - `data_creazione (datetime)`
 - `ultimo_messaggio (varchar(255))`
 - `ultimo_aggiornamento (datetime)`
- Relazioni:
 - 1–N con `Messaggio`

Messaggio

Rappresenta un messaggio scambiato all'interno di una conversazione.

- Chiave primaria: `id`
- Chiave esterna:
 - `conversazione_id` → `Conversazione(id)`
- Attributi:
 - `testo (text)`
 - `timestamp (datetime)`
 - `letto (bit)`
 - `mittente (enum: PAZIENTE, MEDICO)`

DisponibilitaMedico

Rappresenta le fasce orarie settimanali rese disponibili dal medico.

- Chiave primaria: `id`
- Chiave esterna:

- `medico_id` → `Medico(id)`
- **Attributi:**
 - `giorno_settimana` (`varchar(10)`)
 - `ora_inizio` (`time`)
 - `ora_fine` (`time`)
- **Relazioni:**
 - 1–N con `SlotOrario`

SlotOrario

Rappresenta uno slot temporale prenotabile appartenente a una disponibilità del medico.

- Chiave primaria: `id`
- Chiave esterna:
 - `disponibilita_id` → `DisponibilitaMedico(id)`
- **Attributi:**
 - `inizio` (`datetime`)
 - `fine` (`datetime`)
 - `prenotato` (`bit`)

3.5 Controllo Accessi e Sicurezza

Il sistema CardioCare360 prevede un controllo degli accessi basato sui ruoli degli utenti. Ogni utente autenticato può accedere esclusivamente alle funzionalità previste per il proprio ruolo.

Sono previsti tre principali ruoli applicativi:

- **Paziente**
- **Medico**
- **Amministratore**

Il controllo degli accessi consente di separare le responsabilità e di limitare l'accesso ai dati e alle funzionalità in base al ruolo dell'utente.

3.5.1 Gestione dei ruoli

Il ruolo dell'utente è memorizzato nell'entità `Utente` tramite l'attributo `ruolo`, che può assumere i valori:

- `PAZIENTE`
- `MEDICO`

- ADMIN

L'autenticazione consente al sistema di identificare l'utente e il relativo ruolo, permettendo di applicare i permessi associati alle diverse funzionalità.

La separazione dei ruoli è particolarmente importante per la protezione dei dati clinici, in quanto le informazioni relative ai pazienti devono essere accessibili solamente agli utenti autorizzati.

3.5.2 Matrice dei Permessi

La seguente matrice descrive le principali operazioni consentite ai diversi ruoli del sistema

Funzione / Oggetto	Paziente	Medico	Amministratore
Autenticazione	Login / Logout	Login / Logout	Login / Logout
Gestione Profilo	Modifica solo il proprio profilo	Modifica solo il proprio profilo	Modifica solo il proprio profilo
Gestione Utenti	—	—	Crea / Modifica / Elimina
Appuntamenti	Prenota / Visualizza / Annulla	Visualizza / Conferma / Aggiorna	—
Disponibilità Medico	—	Gestisce	—
Esami	Visualizza propri	Gestisce	—
Referti	Visualizza propri	Carica / Modifica	—
Terapie	Visualizza proprie	Gestisce	—

Funzione / Oggetto	Paziente	Medico	Amministratore
Parametri Clinici	Inserisce / Visualizza	Visualizza	—
Storico Parametri	Visualizza	Visualizza	—
Soglie Parametri	Visualizza	Visualizza	—
Notifiche	Visualizza proprie	Visualizza proprie	—
Conversazioni / Messaggi	Chat con medico	Chat con paziente	—

3.5.3 Principi di sicurezza

Il controllo degli accessi è finalizzato a garantire che:

- ogni utente possa accedere alle sole funzionalità previste dal proprio ruolo;
- un **Paziente** possa consultare e gestire esclusivamente le proprie informazioni;
- un **Medico** possa gestire le informazioni cliniche relative ai pazienti assegnati e le funzionalità di propria competenza;
- l'**Amministratore** possa gestire gli account degli utenti;
- le informazioni cliniche siano protette da accessi non autorizzati.

3.6 Controllo Flusso Globale del Sistema

Il sistema **CardioCare360** gestisce funzionalità che richiedono un'interazione continua tra utente e applicazione. Il controllo del flusso è organizzato secondo due modalità complementari:

- **Event-Driven** per il front-end;
- **gestione concorrente delle richieste** per il back-end.

Questo approccio consente di gestire le interazioni dell'utente, le richieste verso i servizi applicativi e le operazioni di accesso ai dati in maniera strutturata e reattiva.

Front-End: Architettura Event-Driven

Nel front-end, realizzato mediante **React**, le interazioni dell'utente sono gestite attraverso eventi associati ai componenti dell'interfaccia.

Esempi di eventi sono:

- click sul comando "**Prenota Appuntamento**";
- selezione di un medico;
- selezione di una data o di uno slot orario;
- caricamento o visualizzazione di un referto;
- modifica del profilo;
- inserimento di un parametro clinico.

L'evento attiva il relativo gestore, che esegue le operazioni necessarie e, quando richiesto, effettua una chiamata alle API REST del back-end.

Il flusso può essere sintetizzato come segue:

1. l'utente interagisce con l'interfaccia;
2. il componente React intercetta l'evento;
3. vengono effettuate le opportune verifiche lato client;
4. viene inviata una richiesta al back-end tramite API REST;

5. il front-end riceve la risposta;
6. lo stato dell'interfaccia viene aggiornato.

Questo modello permette di mantenere separata la gestione dell'interfaccia dalla logica applicativa del back-end.

Back-End: Gestione Concorrente delle Richieste

Il back-end di **CardioCare360**, realizzato mediante **Spring Boot**, espone API REST attraverso i controller applicativi.

Le richieste provenienti dal front-end vengono gestite concorrentemente dall'applicazione, consentendo al sistema di servire più richieste provenienti da utenti differenti.

Il flusso tipico di una richiesta REST è il seguente:

1. il **Controller** riceve la richiesta HTTP;
2. la richiesta viene inoltrata al **Service** corrispondente;
3. il Service esegue la logica di business;
4. quando necessario, il Service interagisce con il livello di persistenza attraverso i repository;
5. il database viene consultato o aggiornato;
6. il risultato viene restituito al Controller;
7. il Controller produce la risposta HTTP, generalmente in formato JSON;
8. il front-end aggiorna l'interfaccia sulla base della risposta ricevuta.

Questo meccanismo consente di gestire contemporaneamente richieste relative, ad esempio, a:

- prenotazione e gestione degli appuntamenti;
- gestione delle disponibilità dei medici;
- consultazione degli esami e dei referti;
- gestione delle terapie;
- inserimento e consultazione dei parametri clinici;
- comunicazione tra paziente e medico.

Flusso Globale del Sistema

Il flusso generale di elaborazione può essere rappresentato attraverso le seguenti fasi:

Utente → React → API REST → Controller → Service → Repository → MySQL

e, al ritorno:

MySQL → Repository → Service → Controller → API REST → React → Utente

In particolare:

1. **Interazione dell'utente**

L'utente esegue un'operazione attraverso l'interfaccia React.

2. **Gestione dell'evento**

Il front-end intercetta l'evento e prepara la richiesta.

3. **Chiamata REST**

React invia la richiesta al back-end.

4. **Controllo della richiesta**

Il Controller riceve la richiesta e la indirizza al servizio appropriato.

5. **Elaborazione della logica di business**

Il Service esegue le operazioni previste dal caso d'uso.

6. **Accesso ai dati**

Se necessario, il Service utilizza i repository per leggere o modificare i dati persistenti.

7. **Risposta del back-end**

Il risultato dell'elaborazione viene restituito tramite una risposta HTTP/JSON.

8. **Aggiornamento dell'interfaccia**

React aggiorna lo stato e la visualizzazione in funzione della risposta ricevuta.

Conclusione

Il controllo del flusso di **CardioCare360** combina la gestione **event-driven** delle interazioni nel front-end con la gestione concorrente delle richieste nel back-end.

La separazione tra interfaccia, API REST, logica di business e persistenza permette di mantenere il sistema modulare e facilita la gestione delle diverse operazioni applicative, tra cui appuntamenti, esami, referti, terapie, parametri clinici e comunicazioni.

3.7 Boundary Conditions

Questa sezione descrive il comportamento del sistema **CardioCare360** nelle condizioni limite, con particolare riferimento alle procedure di avvio e terminazione dell'applicazione e alla gestione delle situazioni di errore.

Le condizioni considerate riguardano:

- l'avvio del sistema;
- la terminazione del sistema;
- i principali scenari di malfunzionamento;
- le modalità di gestione e recupero dagli errori.

3.7.1 Start up

Durante la fase di avvio, i componenti del sistema vengono inizializzati e predisposti per consentire l'accesso alle funzionalità applicative.

Il processo di avvio prevede:

1. avvio del **backend Spring Boot**;
2. inizializzazione dei componenti applicativi;
3. configurazione della connessione al database **MySQL**;
4. avvio del server applicativo e delle API REST;
5. avvio del **frontend React**;
6. disponibilità dell'interfaccia per l'utente.

Una volta completata l'inizializzazione, il sistema è pronto a ricevere le richieste provenienti dal frontend.

3.7.2 Shut down

La terminazione del sistema consiste nell'arresto controllato dei componenti applicativi.

La procedura prevede:

1. interruzione dell'accesso a nuove richieste;
2. completamento delle operazioni attualmente in esecuzione, ove possibile;
3. terminazione del backend;
4. chiusura delle connessioni alle risorse utilizzate;
5. arresto del frontend.

L'arresto controllato ha lo scopo di evitare, per quanto possibile, la perdita o la corruzione dei dati durante la terminazione dell'applicazione.

3.7.3 Failures

Durante l'esecuzione possono verificarsi condizioni di errore che impediscono il completamento di una determinata operazione.

I principali scenari considerati sono:

- **errore di autenticazione**, quando le credenziali fornite non risultano valide;
- **errore di autorizzazione**, quando l'utente tenta di accedere a una funzionalità non prevista dal proprio ruolo;
- **errore di validazione**, quando i dati forniti non rispettano i vincoli richiesti;
- **errore di comunicazione**, quando il frontend non riesce a comunicare correttamente con il backend;
- **errore di persistenza**, quando un'operazione sul database non può essere completata;
- **risorsa non disponibile**, quando una risorsa richiesta non è presente o non può essere recuperata.

In presenza di un errore, il sistema deve evitare, ove possibile, di interrompere l'intera applicazione e deve restituire al componente chiamante un'informazione relativa all'esito dell'operazione.

3.7.4 Gestione dei fallimenti

La gestione dei fallimenti è finalizzata a mantenere il sistema in uno stato consistente e a fornire un feedback comprensibile all'utente.

In particolare:

- gli errori vengono intercettati dal livello applicativo appropriato;
- le operazioni non completate non devono produrre modifiche inconsistenti sui dati;
- il backend restituisce una risposta HTTP coerente con il tipo di errore verificatosi;
- il frontend interpreta la risposta e visualizza un messaggio appropriato;
- l'utente può ripetere l'operazione quando l'errore è temporaneo o correggibile;
- gli errori relativi alla persistenza non devono compromettere la consistenza del database.

Per le operazioni che modificano i dati persistenti, il sistema deve garantire che l'operazione venga completata correttamente oppure che le modifiche parziali vengano annullate quando previsto dal meccanismo transazionale.

Conclusione

La gestione delle **Boundary Conditions** consente a CardioCare360 di mantenere un comportamento controllato anche durante le fasi di avvio, terminazione e in presenza di condizioni anomale.

L'obiettivo è garantire **consistenza dei dati, continuità del servizio e feedback adeguato all'utente**, evitando che un errore relativo a una singola operazione comprometta l'intero sistema.

4. Servizi dei sottosistemi

Di seguito sono riportati tutti i servizi offerti dai sottosistemi di CardioCare360, con relativa descrizione e l'interfaccia applicativa responsabile della loro implementazione.

4.1 Autenticazione

Servizio	Descrizione	Interfaccia
Login	Permette a Paziente, Medico e Amministratore di accedere al sistema tramite credenziali.	AutenticazioneService
Logout	Permette all'utente autenticato di terminare la propria sessione.	AutenticazioneService
Visualizzazione Area Personale	Permette all'utente autenticato di visualizzare i propri dati personali.	AutenticazioneService
Modifica Dati Personali	Permette all'utente di aggiornare i propri dati personali.	AutenticazioneService

4.2 Gestione Appuntamenti

Servizio	Descrizione	Interfaccia
Prenotazione Appuntamento	Il Paziente prenota un appuntamento scegliendo il medico, la data e slot disponibile.	AppuntamentoService
Visualizzazione Appuntamenti	Paziente e Medico visualizzano gli appuntamenti programmati.	AppuntamentoService
Annullamento Appuntamento	Il Paziente annulla un appuntamento precedentemente prenotato.	AppuntamentoService
Aggiornamento Stato Appuntamento	Il Medico aggiorna lo stato dell'appuntamento in funzione dell'esito della prestazione	AppuntamentoService

4.3 Gestione Disponibilità Medico

Servizio	Descrizione	Interfaccia
Inserimento Disponibilità	Il Medico inserisce le proprie fasce orarie disponibili.	DisponibilitaService
Modifica Disponibilità	Il Medico aggiorna le disponibilità precedentemente inserite.	DisponibilitaService
Eliminazione Disponibilità	Il Medico rimuove una disponibilità precedentemente configurata.	DisponibilitaService
Visualizzazione Disponibilità	Il Paziente visualizza le disponibilità dei medici per effettuare una prenotazione.	DisponibilitaService

4.4 Gestione Terapie

Servizio	Descrizione	Interfaccia
Assegnazione Terapia	Il Medico assegna una terapia al Paziente.	TerapiaService
Modifica Terapia	Il Medico aggiorna una terapia precedentemente assegnata.	TerapiaService
Visualizzazione Terapie	Il Paziente visualizza le terapie a lui assegnate.	TerapiaService

4.5 Gestione Conversazioni

Servizio	Descrizione	Interfaccia
Creazione Conversazione	Il sistema crea una conversazione tra Paziente e Medico quando viene avviata una nuova comunicazione.	ConversazioneService
Invio Messaggio	Permette a Paziente e Medico di inviare messaggi all'interno di una conversazione.	MessaggioService
Visualizzazione Messaggi	Permette a Paziente e Medico di visualizzare i messaggi appartenenti a una conversazione.	MessaggioService
Visualizzazione Conversazioni	Permette a Paziente e Medico di visualizzare l'elenco delle conversazioni disponibili.	ConversazioneService
Aggiornamento Stato Messaggio	Il sistema aggiorna lo stato del messaggio (inviato, consegnato, letto).	MessaggioService

4.6 Gestione Utenti

Servizio	Descrizione	Interfaccia
Visualizzazione Lista Utenti	L'Amministratore visualizza tutti gli utenti registrati nel sistema.	GestioneUtentiService
Creazione Utente	L'Amministratore crea un nuovo account utente (Paziente o Medico).	GestioneUtentiService
Modifica Utente	L'Amministratore modifica i dati di un utente.	GestioneUtentiService
Eliminazione Utente	L'Amministratore elimina un utente dal sistema.	GestioneUtentiService

4.7 Gestione Parametri Clinici

Servizio	Descrizione	Interfaccia
Inserimento Parametri Vitali	Il Paziente inserisce i propri parametri clinici rilevati (pressione, frequenza cardiaca, glicemia, ecc.).	ParametriVitaliService
Visualizzazione Parametri Vitali	Paziente e Medico visualizzano l'andamento dei parametri nel tempo.	ParametriVitaliService
Visualizzazione Storico Parametri	Permette di consultare lo storico dei parametri registrati.	ParametriVitaliService
Consultazione Soglie Parametri	Il sistema confronta i valori inseriti dal Paziente con le soglie configurate, generando un alert in caso di superamento	ParametriVitaliService

4.8 Gestione Appuntamento

Servizio	Descrizione	Interfaccia
Registrazione Esito Appuntamento	Il Medico registra le informazioni relative alla prestazione effettuata nell'ambito dell'appuntamento.	VisitaService
Visualizzazione Visite	Paziente e Medico visualizzano gli appuntamenti precedentemente effettuati.	VisitaService

4.9 Gestione Esami

Servizio	Descrizione	Interfaccia
Prescrizione Esame	Il Medico prescrive un esame al Paziente.	EsameService
Visualizzazione Esami	Paziente e Medico visualizzano gli esami prescritti ed effettuati.	EsameService
Aggiornamento Stato Esame	Il Medico aggiorna lo stato dell'esame (programmato, eseguito, annullato).	EsameService

4.10 Gestione Referti

Servizio	Descrizione	Interfaccia
Caricamento Referto	Il Medico carica il referto relativo a un esame.	RefertoService
Modifica Referto	Il Medico modifica un referto precedentemente inserito.	RefertoService
Visualizzazione Referti	Il Paziente visualizza i referti associati ai propri esami.	RefertoService
Generazione Referto PDF	Il sistema genera il referto in formato PDF scaricabile.	RefertoService

5. Glossario

Termine	Definizione
CardioCare360	Applicativo web per la gestione digitale di appuntamenti, esami, referti, terapie, parametri clinici e comunicazioni tra Paziente e Medico.
Paziente	Utente che prenota appuntamenti, visualizza esami, referti, terapie e parametri clinici e comunica con il Medico.
Medico	Utente che gestisce disponibilità, appuntamenti, esami, referti, terapie e comunica con i Pazienti.
Amministratore	Utente che gestisce gli account del sistema e le relative operazioni amministrative.
Appuntamento	Incontro programmato tra Paziente e Medico, con data, ora, tipo di prestazione e stato associati.
Disponibilità	Giorni e fasce orarie in cui un Medico è disponibile per ricevere appuntamenti.
Slot Orario	Intervallo temporale prenotabile all'interno della disponibilità del Medico.
Esame	Prestazione diagnostica associata a un Paziente e a un Medico, con data, ora, tipo e stato.
Referto	Documento clinico redatto dal Medico a seguito di un esame e associato al relativo Paziente.
Terapia	Trattamento farmacologico o terapeutico assegnato dal Medico al Paziente.
Parametro Clinico	Misurazione sanitaria registrata per un Paziente, utilizzata per il monitoraggio delle sue condizioni cliniche.

Termine	Definizione
Storico Parametri	Archivio cronologico dei parametri clinici registrati per un Paziente.
Soglia Parametro	Valore limite associato a un parametro clinico per un determinato paziente, utilizzato dal sistema per individuare condizioni di attenzione.
Conversazione	Canale di comunicazione tra Paziente e Medico.
Messaggio	Comunicazione testuale scambiata all'interno di una conversazione.
Backend	Parte del sistema che gestisce la logica applicativa, i servizi, le API REST e l'accesso al database.
Frontend	Interfaccia grafica attraverso la quale l'utente interagisce con il sistema.
Database	Sistema di memorizzazione dei dati persistenti relativi a utenti, appuntamenti, esami, referti, terapie e parametri clinici.
API REST	Interfacce che permettono la comunicazione tra frontend e backend tramite richieste HTTP.
Autenticazione	Processo che consente di verificare l'identità di un utente prima dell'accesso al sistema.
Autorizzazione	Processo che determina quali funzionalità e risorse un utente può utilizzare in base al proprio ruolo.
JWT	JSON Web Token utilizzato dal sistema per rappresentare le informazioni necessarie alla gestione dell'autenticazione dell'utente.
Backup	Copia dei dati persistenti utilizzata per consentire il ripristino delle informazioni in caso di guasti o perdita dei dati.
Persistenza	Memorizzazione stabile dei dati nel database.

Termine	Definizione
COTS	<i>Commercial Off-The-Shelf</i> : componente software o hardware disponibile sul mercato e utilizzabile all'interno del progetto.
VARCHAR	Tipo di dato SQL utilizzato per memorizzare stringhe di lunghezza variabile.
INTEGER	Tipo di dato SQL utilizzato per memorizzare valori numerici interi.
DATE	Tipo di dato SQL utilizzato per rappresentare una data.